Industrijska informatika – Laboratorijske vaje

študijsko leto 2026/27

2. vaja: **Načrtovanje logičnih krmilj z diagrami stanj in simulacijo**

Pri izvajanju proizvodnih procesov igrajo ključno vlogo programirljiva logična krmilja, ki omogočajo avtomatizacijo proizvodnih postopkov. Z naraščajočo kompleksnostjo proizvodnih sistemov narašča tudi zahtevnost razvoja ustrezne programske opreme.

Diagrami stanj (Statecharts, UML State Machines) omogočajo v povezavi s ustreznimi programskimi orodji, kot je MATLAB & Simulink Stateflow, natančno specifikacijo sekvenčnih logičnih krmilj, simulacijo delovanja takega krmilja in tudi generiranje kode za PLK.

MATLAB & Simulink Stateflow

V okviru simulacijskega orodja Simulink je na voljo dodatna knjižnico Stateflow, ki razširja funkcionalnost orodja Simulink. Knjižnica Stateflow implementira grafični jezik, ki vključuje diagrame stanj, diagrame poteka, tabele prehodov stanj in pravilnostne tabele. Stateflow lahko uporabimo za opis odzivanja Matlabovih algoritmov in Simulinkovih simulacijskih modelov na vhodne signale, dogodke in časovno odvisne pogoje.

Z orodjem Stateflow lahko formuliramo kombinacijsko ali sekvenčno odločitveno logiko, ki jo je mogoče simulirati kot blok v Simulinku ali izvajati kot objekt v Matlabu. Grafična animacija omogoča analizo in odpravljanje napak logike med izvajanjem.

Stateflow omogoča načrtovanje in razvoj krmilne logike, nadzornih sistemov, logike razvrščanja opravil, upravljanje napak, razvoj komunikacijskih protokolov, uporabniških vmesnikov in hibridnih sistemov.

Modularna zasnova programa PLK in kombiniranje programskih jezikov

Standard IEC 61131-3 definira štiri osnovne jezike programirljivih logičnih krmilnikov (PLK), s katerimi enostavno programiramo logične in aritmetične operacije. To so jeziki IL (Instruction List) – ukazna lista, ST (Structured Text) – strukturiran tekst, LD (Ladder Diagram) – lestvični diagram in FBD (Function Block Diagram) – funkcijski blokovni diagram.

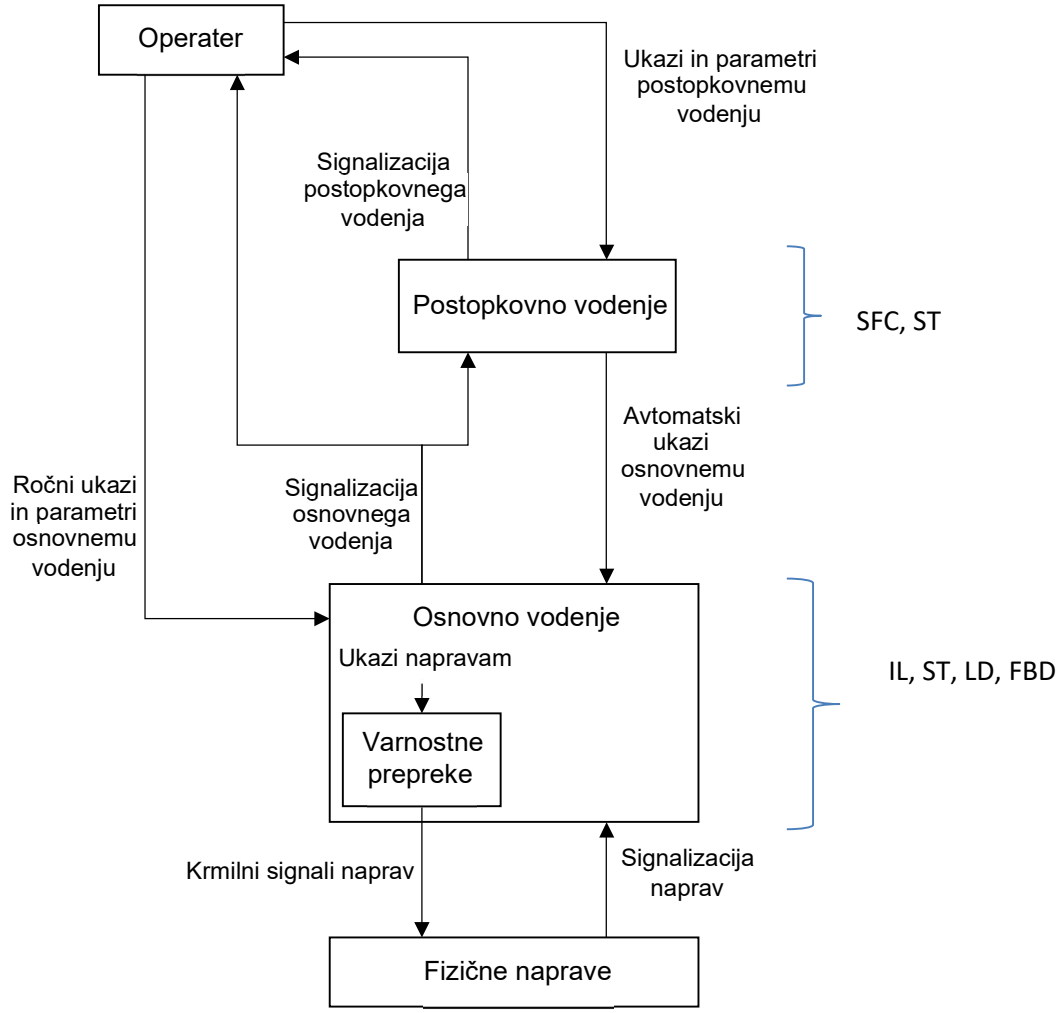
Standard definira tudi dodatni višjenivojski grafični programski jezik, sekvenčni funkcijski diagram (SFC), ki je prilagojen natančni specifikaciji in programiranju postopkov. Alternativno se za specifikacijo in programiranje postopkov uporablja tudi strukturiran tekst (ST), ki je višjenivojski programski jezik.

Višjenivojska jezika SFC in ST sta manj primerna za programiranje osnovnih funkcij vodenja, kot so npr. varnostne prepreke, signalizacija in interakcija z operaterjem. Zato največkrat

kombiniramo dele krmilnega programa, ki so napisani v sekvenčnem funkcijskem diagramu ali v strukturiranem tekstu, z deli krmilnega programa v drugih programskih jezikih (IL, LD, FBD), kot kaže slika 1. Če vsebuje osnovno vodenje obsežnejše matematične izračune, npr. pri regulacijah, tudi v tem delu uporabljamo jezik ST.

Takšna modularna zasnova programov ima tudi druge prednosti, od lažjega razvoja programov in lažje diagnostike do možnosti ponovne uporabe razvitih modulov v novih programih.

V vaji bomo preizkusili specifikacijo postopkov s diagrami stanj v okviru na sliki 1 nakazanega modularnega pristopa k programiranju. Postopkovno vodenje bomo preizkušali na simulacijskih modelih postaj laboratorijskega modularnega proizvodnega sistema (MPS) ter na modelu simulatorja dvigala ELWE.



Slika 1: Modularna zasnova in kombiniranje programskih jezikov v programu PLK

Naloge:

1. Za dodeljeno delovno postajo modularnega proizvodnega sistema oz. simulator dvigala specificirajte postopek, ki ga izvede v enem delovnem ciklu v normalnem načinu delovanja. Pomagajte si z opisom delovanja postaj v dokumentih »Modularni proizvodni sistem« in »Simulator dvigala« (dokumenta sta dostopna v spletni učilnici). Narišite ustrezni diagram stanj na papir ali na računalnik s pomočjo ustreznega urejevalnika (npr. PlantText). Pri tem upoštevajte načelo dobre prakse, po katerem postopkovno vodenje neposredno ne spreminja izhodnih signalov krmilnika, temveč le signalizira ustrezne ukaze osnovnemu vodenju preko izhodnih spremenljivk diagrama stanj (slika 1). To se nanaša tudi na zahtevano utripanje lučk – postopkovno vodenje postavi zahtevo za utripanje, dejansko utripanje pa se izvede v okviru osnovnega vodenja.
2. V orodju Simulink odprite pripravljeno simulacijsko predlogo za simulacijo dodeljenega procesa (predloge so dostopne v spletni učilnici). Iz knjižnice Stateflow prenesite blok Chart in ga vstavite v podsistem »Procedural_control« znotraj odprte predloge.
3. V orodju Stateflow definirajte nabor spremenljivk bloka Chart, ki bodo služile za povezavo diagrama stanj s preostalim delom simulacijske sheme. Vse spremenljivke morajo biti binarne (Type: boolean) in skalarne (Size: 1). Izjema je meritev višine na 2. postaji MPS, ki je celo število (Type: int16).
4. V urejevalniku Stateflow narišite diagram stanj iz točke 1, ki predstavlja delovni cikel procesa. Vpišite ustrezne prehodne pogoje in akcije v stanjih diagrama.
5. Povežite diagram stanj s preostalim delom simulacijske sheme. Pri tem upoštevajte zgoraj zapisano načelo, da naj postopkovno vodenje neposredno ne spreminja izhodnih signalov krmilnika, temveč posreduje ukaze (v simulacijski predlogi je takšna povezava že predvidena – opis spodaj).
6. Znotraj podsistema »Basic_control« dopolnite izvedbo varnostnih pogojev in signalizacijo.
7. Preizkusite delovanje krmilnega programa na simulacijskem modelu dodeljenega procesa. Odpravite morebitne napake ali pomanjkljivosti.
8. Dokumentiranje rezultate vaje in izdelajte poročilo, tako da v simulacijsko shemo vpišete komentarje, ki pojasnjujejo delovanje diagrama stanj, ter natisnete simulacijsko shemo, vključno z diagramom stanj, funkcijami osnovnega vodenja in ustreznimi komentarji.

Dodatna pojasnila o izvedbi vaje

Za specifikacijo krmiljenja uporabite vnaprej pripravljene simulacijske predloge v orodju Simulink. Vsaka predloga vsebuje osnovno logiko in preprosto animacijo delovanja posamezne postaje MPS oz. simulatorja dvigala, vhodno/izhodne signale krmilnika ter že pripravljena bloka za specifikacijo postopkovnega in osnovnega vodenja.

Blok za **postopkovno vodenje** (Procedural_control) v predlogi ima dva vhoda in dva izhoda, kar predstavlja štiri sklope vhodno/izhodnih signalov:

- Sen: multipleksirani signali senzorjev simuliranega procesa (na 2. postaji MPS je ločen vhod SenAD za meritev višine),

- UI: multipleksirani signali tipk na komandnem pultu,

- Cmd: multipleksirani signali, ki predstavljajo ukaze osnovnemu vodenju za vklope aktuatorjev,

- OPCmd: multipleksirani signali, ki predstavljajo ukaze osnovnemu vodenju za signalizacijo (število in pomen teh signalov je odvisen od zasnove diagrama stanj).

V okviru **osnovnega vodenja** (Basic_control) je za potrebe vaje sprogramirana možnost preklopa med ročnim in avtomatskim delovanjem, primer izvedbe preslikave ukazov Cmd v krmilne izhode z dodatnimi varnostnimi pogoji ter osnovna signalizacija na podlagi ukazov OPCmd, vključno s primerom utripanja.

Simulacijska predloga vsebuje tudi stikalo za preklop v **ročno vodenje**. Ta možnost je predvidena predvsem za seznanjanje s funkcionalnostjo simulatorja. Če preklopimo stikalo Manual Switch v položaj On, potem signali Cmd postopkovnega vodenja nimajo vpliva, temveč se osnovno vodenje procesa odziva na signale pod oznako Manual commands, ki jih vklapljamo in izklapljamo z miško. Ob tem so še vedno aktivni dodatni varnostnimi pogoji. (Pri realni izvedbi krmiljenja v 4. vaji bomo signale Manual commands generirali s tipkami na krmilnem pultu.)

Z miško lahko tudi dodajamo (ter pri nekaterih postajah MPS odvzemamo) obdelovance preko gumbov pod oznako Workpiece.

V okviru vaje je potrebno v vnaprej pripravljen blok za postopkovno vodenje vstaviti blok Stateflow Chart, definirati in povezati ustrezne vhodno/izhodne signale tega bloka ter s pomočjo urejevalnika Stateflow zgraditi diagram stanj, ki bo realiziral delovanje procesa v skladu z besednim opisom (podan v dokumentih »Modularni proizvodni sistem« in »Simulator dvigala«). V bloku za osnovno vodenje je potrebno dopolniti izvedbo varnostnih pogojev in signalizacijo ter preizkusiti simulirano delovanje krmilnega sistema.

Oznake vhodno/izhodnih signalov simulirane postaje MPS in komandnega pulta so povzete po oznakah v prireditveni tabeli postaje v dokumentu »Modularni proizvodni sistem«. Uporabniško definirane spremenljivke v diagramu stanj (blok Chart) so lahko poljubne in naj odražajo pomen signalov. Paziti je treba na pravilno povezavo vhodov in izhodov bloka Chart s preostankom simulacijske sheme.

Diagrami stanj v orodju Stateflow in modeliranje logičnega krmilja

Spremenljivke

Vse spremenljivke, ki jih uporabljamo v diagramu, morajo biti ustrezno deklarirane kot vhodne, izhodne ali notranje spremenljivke. Za deklaracijo uporabljamo tabelo Symbols, ki se pojavi kot panel ob strani diagrama stanj. Če panela ne vidimo, izberemo zavihek MODELING in nato Symbols Pane. Spremenljivke dodajamo z ukazom Create Data (prva ikona v orodjarni panela), nato pa lahko naknadno določimo, ali gre za izhodno, vhodno ali drugo spremenljivko s klikom na ikono pred imenom spremenljivke in ustrezno izbiro. Tip in velikost spremenljivke lahko spreminjamo, če odpremo Property Inspector, npr. tako, da kliknemo na ime spremenljivke z desnim gumbom in izberemo Inspect…

Druga možnost je, da vse spremenljivke najprej vpišemo v diagram stanj, nato pa kliknemo ikono Resolve undefined symbols (simbol ). Vse nedeklarirane spremenljivke se samodejno dodajo v panel, ročno pa moramo določiti, ali gre za vhod, izhod ali drugo obliko spremenljivke.

Spremenljivke, ki so bile deklarirane kot vhodi ali izhodi bloka, se pojavijo kot vhodno izhodni signali ob straneh simbola bloka v simulacijski shemi.

Prehodi

V diagramu stanj, s katerim modeliramo program PLK, je priporočljivo prehode med stanji vezati zgolj na varovala (condition) in ne na dogodke (events). Varovala zapišemo v obliki logičnega izraza znotraj oglatih oklepajev. Lahko uporabimo poljuben Matlabov izraz, ki vključuje deklarirane spremenljivke in funkcije ter vrne kot rezultat logično vrednost.

Če je spremenljivka deklarirana kot logična spremenljivka, zadošča navedba imena spremenljivke, ni treba primerjati spremenljivke z vrednostjo 'true' (npr. namesto [CLEAR == true] je dovolj zapis [CLEAR].

(Več o prehodih najdemo na https://www.mathworks.com/help/stateflow/ug/transitions.html.)

Akcije

Spremenljivke spreminjamo z akcijami v izbranem akcijskem jeziku (MATLAB ali C), ki ga nastavimo s parametrom Action Language med lastnostmi diagrama (Chart Properties). Izbrani akcijski jezik je ponazorjen s simbolom v spodnjem levem kotu delovnega okna diagrama.

Če je izbran akcijski jezik MATLAB, lahko akcije vsebujejo poljubne Matlabove ukaze. Te lahko pišemo v stanja diagrama pod oznako stanja. Dopustno je uporabljati tudi akcije ob prehodih (pogojne akcije: ukazi znotraj zavitih oklepajev ali prehodne akcije: ukazi za /), vendar je bolj priporočljivo uporabljati akcije znotraj stanj. Tu lahko uporabljamo kvalifikatorje akcij en: (entry), ex: (exit) ali du: (during). Če te oznake ne uporabimo, se akcija izvaja več čas, ko je stanje aktivno (ekvivalent du:).

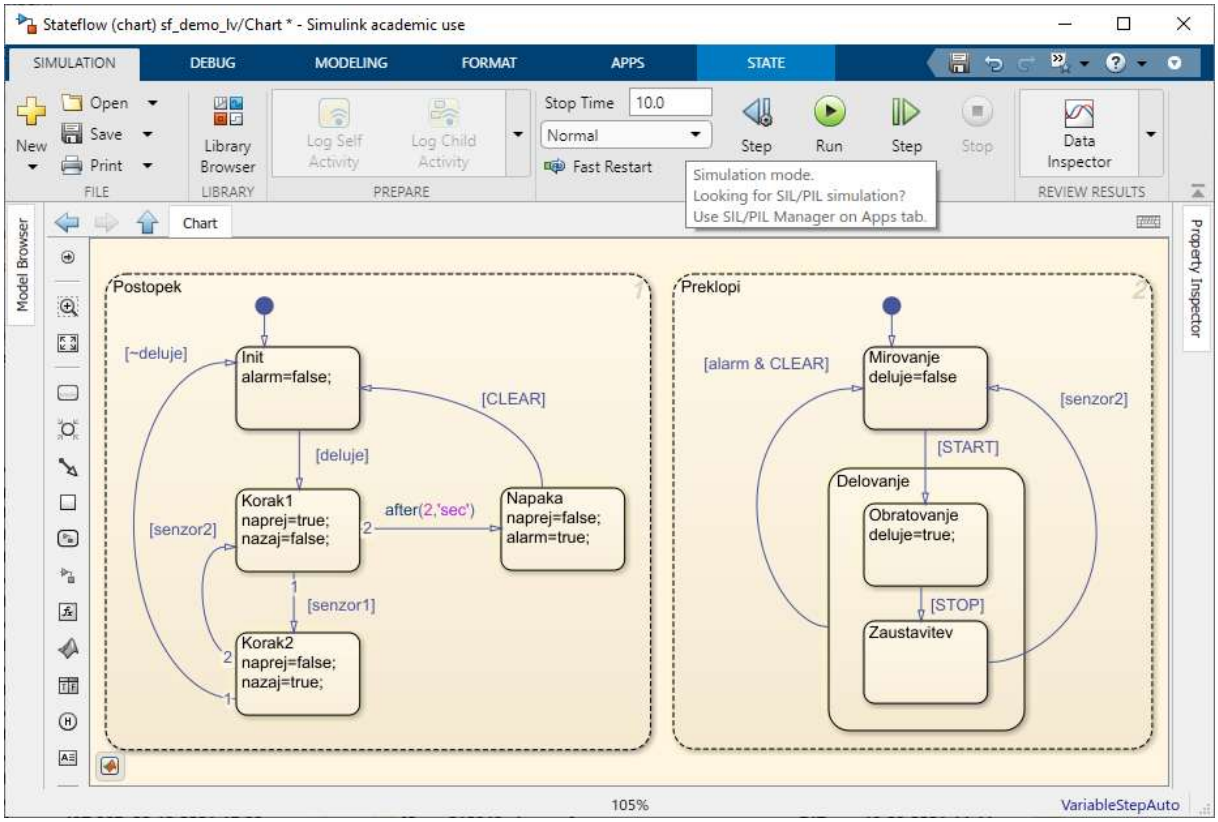
Opomba: pri binarnih spremenljivkah moramo pri prirejanju konstantnih vrednosti uporabljati oznaki true in false.

(Pregled operacij in operatorjev akcijskega jezika najdemo na https://www.mathworks.com/help/stateflow/ug/supported-operations-on-chart-data.html.)

Časovne zakasnitve

Če želimo v diagramu stanj uporabljati časovne zakasnitve, je namesto *duration* bolje uporabiti ključni besedi *after* oz. *before*. Ko bomo v nadaljevanju generirali kodo za programirljivi logični krmilnik, se bo pri generiranju kode poleg izvedbe diagrama stanj v jeziku SCL generiral tudi ustrezen časovnik, ki ga generirani blok uporabi za pravilno trajanje zakasnitev.

Primer diagrama, ki na ta način realizira zakasnitev za namen generiranja alarma, je na sliki 2. Alarm se sproži, če 2 s po začetku akcije za pomik naprej ne dobimo informacije s senzorja 1, da je premik končan.



Slika 2: Primer diagrama stanj s časovnimi zakasnitvami

V zgornjem primeru je bil parameter Update method Stateflow-bloka nastavljen na »inherited«, lahko pa nastavimo tudi »discrete« s fiksnim časom vzorčenja; na pravilnost zakasnitev to ne vpliva.

Vzporedna stanja (ortogonalne regije)

Primer na sliki 2 kaže tudi uporabo vzporednih poddiagramov stanj in nadstanj/podstanj, kar omogoča enostavno izvedbo preklopov med načini delovanja, zagon, zaustavitev ipd.

Če želimo vzporedno izvajanje, moramo zunaj stanj, ki naj bi se izvajala vzporedno, klikniti z desnim gumbom in izbrati Decomposition > Parallel (AND). Vzporedno delujoča stanja spremenijo rob v črtkano črto. Nato lahko znotraj vsakega od takšnih stanj narišemo diagram stanj. Diagrami se bodo izvajali vzporedno, delili pa si bodo vse deklarirane spremenljivke.

V primeru na sliki je vzporedno stanje Preklopi namenjeno preklapljanju med načini delovanja. Razvidna je tudi skupna raba nekaterih spremenljivk v obeh vzporedno delujočih poddiagramih stanj.